



Fonctions et Procédures





Fonction : Définition

- Une Fonction est un groupe d'instructions qui assurent ensemble une ou plusieurs tâches. Chaque programme C a au moins une fonction, qui est `main()`.
- Vous pouvez diviser votre code en des fonctions séparées. La logique de la division est que chaque fonction assure une tâche spécifique.
- La déclaration de la fonction donne au compilateur le nom de la fonction, le type de retour, et les paramètres. La définition d'une fonction fournit le corps réel de la fonction.
- La bibliothèque standard C fournit un certain nombre de fonctions prédéfinies que votre programme peut appeler.
- Par exemple, la fonction `strcat()` pour concaténer deux strings, et bien d'autres fonctions.
- Une fonction est connue sous plusieurs noms, comme: une méthode ou une procédure ou sub-routine, etc.



fonction : Définition

- La forme générale de la definition d'une fonction en C est comme suit:

```
return_type nom_fonction( liste_de_parametres ) {  
    Corps de la fonction  
}
```

Une définition de fonction en C consiste a un entête de fonction et un corps de fonction. Here are all the parts of a fonction:

- **Type de Retour**: Une fonction peut retourner une valeur. Le type de retour est un type de données que la fonction retourne. Certaines fonctions effectuent les opérations désirées sans retourner de valeurs. Dans ce cas, le type de retour est le mot-clé [void](#).
- **Nom de la Fonction**: ceci est le nom réel d'une fonction. Le nom de la fonction et la liste de paramètres ensemble constituent la [signature](#) de la fonction.
- **Paramètres**: un paramètre est comme un placeholder. Quand une fonction est invoquée, vous passer une valeur au paramètre. Cette valeur consiste a un paramètre ou argument. La liste de paramètres consiste au type, ordre, et nombre de paramètres d'une fonction. Les paramètres sont optionnels; une fonction peut être défini paramètres.
- **Corps de la Fonction**: le corps de la fonction contient une collection d'instructions qui définit ce que la fonction fait.



xemple

- Le suivant est un code source pour une fonction nommée **max()**. Cette fonction prend deux paramètres num1 et num2 et retourne le maximum entre eux:

```
/* function returning the max between two numbers */  
int max(int num1, int num2) {  
    /* local variable declaration */  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```



déclaration d'une Fonction

- La déclaration d'une fonction donne au compilateur le nom de la fonction et comment l'appeler. Le corps réel de la fonction peut être défini après la déclaration « **Fonction Prototype** ».

- La déclaration d'une fonction consiste au éléments suivants:

```
return_type nom_fonction( liste de parametres );
```

- Pour la fonction **max()**, la ligne suivante représente sa déclaration:

```
int max(int num1, int num2);
```

- Les noms de Paramètres ne sont pas importants dans la declaration d'une fonction. Au contraire, le type de retour est requis, donc la déclaration suivante est valide:

```
int max(int , int );
```



appel de Fonction

- En creant une fonction, vous donner une definition pour effectuer une tache.
- Quand vous appeler une fonction, le controle du programme est transferé a la fonction, pour appliquer les données fournies a l'ensemble d'instructions dans cette fonction.
- Pour appeler une fonction, vous devez tout simplement passer les parametres requis, nom et le type de retour.



appel à Fonction (Exemple)

```
#include <stdio.h>
/* declaration d'une fonction */
int max(int num1, int num2);
int main () {
/* declaration de variable local */
int a = 100; int b = 200; int ret;
/* appel a une fonction pour retourner le
max */
ret = max(a, b);
printf( "Max est : %d\n", ret );
return 0;
}
```

```
/* fonction qui retourne le max de
deux nombres */
int max(int num1, int num2) {
/* declaration de variable local */
int result;
if (num1 > num2)
result = num1;
else
result = num2;
return result;
}
```



appel á Fonction (Exemple)

```
/* swap : echange de valeurs */  
void swap(int x, int y) {  
    int temp;  
    temp = x;  
    x = y;  
    y = temp;  
}
```

```
#include <stdio.h>  
void swap(int x, int y);  
int main () {  
    int a = 100; int b = 200;  
    printf("avant le swap, valeur de a : %d\n", a );  
    printf("avant le swap, valeur de b : %d\n", b );  
    /* appel á la fonction swap */  
    swap(a, b);  
    printf("apres le swap, valeur de a : %d\n", a );  
    printf("apres le swap, valeur de b : %d\n", b );  
    return 0;  
}
```